

Databases 101 — Relational

Phase 1 · Fundamentals · Estimated time: 2 hours

1. Concept From First Principles

English:

You've used MySQL/PostgreSQL for 9 years, so this chapter reframes what you already know through an HLD lens: not 'how do I write this query' but 'why does this database behave the way it does under load, and what are its guarantees worth in an architecture conversation.'

Hindi:

आप 9 साल से MySQL/PostgreSQL use करते आए हैं, तो यह chapter उसी जानकारी को HLD नज़रिए से दोबारा दिखाता है: सवाल यह नहीं है 'यह query कैसे लिखूं' बल्कि यह है 'load में यह database ऐसा behave क्यों करता है, और architecture की बातचीत में इसकी guarantees की कीमत क्या है।'

Important Words:

- **Reframe** – फरि से समझना / नए नज़रिए से देखना

What makes a database "relational"

English:

It's worth being explicit about why this matters for HLD specifically: choosing a relational database is itself an architecture decision with consequences you'll be expected to defend, not a neutral default. Every constraint the database enforces is a correctness guarantee you get for free.

Hindi:

यह साफ समझना ज़रूरी है कि HLD के लिए यह क्यों मायने रखता है: relational database चुनना खुद एक architecture decision है जिसके नतीजों को आपको defend करना पड़ेगा, यह कोई neutral default नहीं है। database जो भी constraint लागू करता है, वह आपको मुफ्त में मिली एक correctness guarantee है।

Important Words:

- **Constraint** – प्रतिबंध / शर्त

English:

A Relational Database Management System (RDBMS) organizes data into tables with fixed schemas, connected by relationships (foreign keys), and queried with SQL. The defining strength is that it enforces structure and correctness rules inside the database itself.

Hindi:

एक Relational Database Management System (RDBMS) data को tables में organize करता है जिनकी schema तय होती है, जो relationships (foreign keys) से जुड़े होते हैं, और SQL से query किए जाते हैं। इसकी सबसे बड़ी ताकत यह है कि यह structure और correctness के नियम खुद database के अंदर ही लागू करता है।

Important Words:

- **Schema** – स्कीमा / संरचना
- **Foreign key** – फॉरेन की

ACID -- the guarantee that makes RDBMS trustworthy

English:

- Atomicity: a transaction either fully completes or fully rolls back.
- Consistency: a transaction takes the database from one valid state to another.
- Isolation: concurrent transactions don't see each other's uncommitted changes.
- Durability: once committed, data survives a crash.

Hindi:

- Atomicity: एक transaction या तो पूरी तरह complete होती है या पूरी तरह rollback हो जाती है।
- Consistency: transaction database को एक valid state से दूसरी valid state में ले जाती है।
- Isolation: एक साथ चल रही transactions एक-दूसरे के uncommitted बदलाव नहीं देख पाती।
- Durability: commit होने के बाद, data crash के बाद भी सुरक्षित रहता है।

Important Words:

- **Rollback** – रोलबैक / वापस लौटाना
- **Concurrent** – समवर्ती / एक साथ चलने वाला
- **Committed** – कमिट किया गया

Backend engineer analogy

English:

Every time you wrap a wallet-debit and order-creation inside a Laravel DB::transaction() block, you're relying on ACID directly. If the debit succeeds but order-creation throws, atomicity guarantees the debit rolls back too.

Hindi:

जब भी आप wallet-debit और order-creation को Laravel के DB::transaction() block में लपेटते हैं, तो आप सीधे ACID पर भरोसा कर रहे होते हैं। अगर debit हो जाए लेकिन order-creation में error आ जाए, तो atomicity गारंटी देती है कि debit भी वापस (rollback) हो जाएगा।

Indexing -- why some queries are instant and others aren't

English:

Without an index, the database scans every row to find a match -- fine for 1,000 rows, catastrophic for 500 million. An index is a separate, sorted structure that lets the database jump directly to matching rows. The cost: every index speeds up reads but slows down writes.

Hindi:

बना index के, database हर row को check करके match ढूंढता है -- 1,000 rows के लिए ठीक है, लेकिन 50 करोड़ rows के लिए बहुत भारी पड़ता है। एक index एक अलग, sorted structure होता है जो database को सीधे matching rows तक पहुंचाता है। इसकी कीमत यह है: हर index reads को तेज़ करता है लेकिन writes को धीमा।

Important Words:

- **Catastrophic** – वनाशकारी / बहुत बुरा

English:

Isolation deserves one more layer of nuance. Databases offer multiple isolation levels, each allowing progressively fewer concurrency anomalies at the cost of more locking. Most production systems default to Read Committed, not the strictest Serializable level.

Hindi:

Isolation के बारे में एक और बारीक बात समझनी ज़रूरी है। Databases कई isolation levels देते हैं, हर एक level ज्यादा locking की कीमत पर कम concurrency anomalies रोकता है। ज्यादातर production systems सबसे सख्त Serializable level नहीं बल्कि Read Committed को default रखते हैं।

Important Words:

- **Anomaly** – वसिंगति / गड़बड़ी
- **Locking** – लॉकगि

2. Real-World Example / Case Study

English:

Razorpay's core ledger -- the system of record for every rupee moving through their platform -- runs on a relational database, precisely because ACID guarantees are non-negotiable for money: a payment must never be 'half-recorded.'

Hindi:

Razorpay का core ledger -- जो उनके platform से गुज़रने वाले हर रुपये का record रखता है -- एक relational database पर चलता है, ठीक इसलिए क्योंकि पैसे के लिए ACID guarantees में कोई समझौता नहीं हो सकता: एक payment कभी 'आधा-रिकॉर्ड' नहीं होनी चाहिए।

Important Words:

- **Non-negotiable** – जसि पर समझौता ना हो सके

English:

Flipkart's order and catalog systems historically use relational databases for the same reason: an order record has strict relationships that map naturally onto foreign keys and transactions.

Hindi:

Flipkart के order और catalog systems भी इसी वजह से relational databases use करते आए हैं: एक order record के strict relationships होते हैं जो foreign keys और transactions पर स्वाभाविक रूप से फिट बैठते हैं।

English:

A second example: PhonePe's KYC verification records use foreign keys tying document metadata to a customer's account row, with unique constraints preventing two accounts from claiming the same ID number.

Hindi:

एक और उदाहरण: PhonePe के KYC verification records foreign keys का उपयोग करते हैं जो document की metadata को customer के account row से जोड़ते हैं, साथ ही unique constraints यह रोकते हैं कि दो accounts एक ही ID number claim ना करें।

3. Diagram (English labels kept as-is)

Without an index (full table scan):
 SELECT * FROM orders WHERE user_id = 4521;
 -> DB checks EVERY row in the table, one by one $O(n)$

With an index on user_id:
 orders_user_id_idx (B-Tree, sorted by user_id)
 4521 -> row ptr <-- jumps straight here
 -> DB jumps directly to matching rows $O(\log n)$

Trade-off: every INSERT/UPDATE must also update this index.

4. Trade-offs Table (Reference Table – English)

Decision	Benefit	Cost
Add an index on a column	Much faster reads on that column	Slower writes, extra storage
Wrap operations in a transaction	Guarantees atomicity/consistency	Locks resources longer
Strict foreign key constraints	Prevents invalid data at the DB layer	Slower writes, rigid changes
Normalize schema heavily	No duplicate data	More JOINS, can hurt read performance

5. Common Interview Questions

Q1: "Why would you choose a relational database for this system?"

English:

Model approach: Tie the answer to the data's actual relationships and correctness needs -- e.g., orders relate strictly to users, items, and payments, and money must never be partially recorded.

Hindi:

Model approach: जवाब को data के असली relationships और correctness की ज़रूरत से जोड़ें -- जैसे orders का users, items, और payments से सख्त relationship होता है, और पैसा कभी आधा record नहीं होना चाहिए।

Q2: "This query is slow at scale -- what would you check first?"

English:

Model approach: Check whether the filtered column has an index, check the query plan, consider whether it's doing a full scan or inefficient JOIN, and only then consider caching or read replicas.

Hindi:

Model approach: check करें कि filter किए गए column पर index है या नहीं, query plan देखें, यह देखें कि किहीं full scan या inefficient JOIN तो नहीं हो रहा, और तभी caching या read replicas के बारे में सोचें।

Q3: "What's the difference between Read Committed and Serializable isolation?"

English:

Model approach: Read Committed only guarantees you never read uncommitted data, but allows some anomalies. Serializable eliminates all anomalies but requires far more locking, hurting throughput.

Hindi:

Model approach: Read Committed सरिफ़ इतनी गारंटी देता है कि आप कभी uncommitted data नहीं पढ़ेंगे, लेकिन कुछ anomalies की इजाज़त देता है। Serializable सभी anomalies खत्म करता है लेकिन इसके लिए बहुत ज़्यादा locking चाहिए, जिससे throughput घटता है।

Important Words:

- **Throughput** – थ्रूपुट / काम करने की दर

6. Practice Exercises

English:

- Explain ACID using a real transaction from an app you've built.
- Explain why 'SELECT * FROM users WHERE email = ?' on 50 million rows needs an index.
- List one scenario where you'd deliberately choose NOT to add an index.

Hindi:

- अपने बनाए किसी app की एक असली transaction से ACID समझाएं।
- समझाएं कि 5 करोड़ rows पर 'SELECT * FROM users WHERE email = ?' को index की ज़रूरत क्यों है।
- एक ऐसा scenario बताएं जहाँ आप जानबूझकर index नहीं जोड़ेंगे।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"ACID Properties in Databases With Examples" — ByteByteGo	Concise, visual.
English	"Database Engineering" series — Hussein Nasser	Goes deep on indexing and internals.
Hindi	"ACID Properties in DBMS" — Well Academy	Structured Hindi explanation.
Hindi	"Lec-87: Introduction to Transaction Concurrency in Hindi" — Gate Smashers	Covers transactions and concurrency.

8. Key Takeaways

English:

- RDBMS enforces structure and correctness inside the database itself.
- ACID = Atomicity, Consistency, Isolation, Durability -- the guarantees that make transactions trustworthy.
- Indexes trade write speed and storage for much faster reads.
- Money, inventory, and anything needing strict correctness defaults to a relational database with ACID.
- Full table scans are fine at small scale and catastrophic at large scale.
- ACID guarantees only apply within a single database instance, not automatically across services.

Hindi:

- RDBMS structure और correctness को खुद database के अंदर ही लागू करता है।

- ACID = Atomicity, Consistency, Isolation, Durability -- यही guarantees transactions को भरोसेमंद बनाती है।
- Indexes write speed और storage की कीमत पर बहुत तेज़ reads देते हैं।
- पैसा, inventory, और सख्त correctness चाहने वाली हर चीज़ ACID वाले relational database की तरफ जाती है।
- Full table scans छोटे scale पर ठीक हैं लेकिन बड़े scale पर वनिशकारी।
- ACID guarantees सिर्फ़ एक database instance के अंदर लागू होती है, services के बीच अपने आप नहीं फैलती।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Anomaly	वसिंगति / गड़बड़ी	An unexpected, incorrect result
Catastrophic	वनिशकारी / बहुत बुरा	Extremely bad or damaging
Committed	कमटि कयिा गया	Permanently saved
Concurrent	समवर्ती / एक साथ चलने वाला	Happening at the same time
Constraint	प्रतबिंध / शर्त	A rule that limits what data is allowed
Foreign key	फॉरेन की	A link from one table's row to another table's row
Locking	लॉकगि	Temporarily blocking access to data during a transaction
Non-negotiable	जसि पर समझौता ना हो सके	Not able to be compromised on
Reframe	फरि से समझना / नए नज़रएि से देखना	To look at the same thing from a different angle
Rollback	रोलबैक / वापस लौटाना	Undoing a partially completed operation
Schema	स्कीमा / संरचना	The defined structure of a table's columns
Throughput	थ्रूपुट / काम करने की दर	The amount of work done per unit time